

TECHNICAL REPORT

Firewood Performance

An evidence-driven study of Firewood performance on representative C-Chain workloads, using replay benchmarks and bottleneck analysis to turn measurements into practical optimization decisions. AI-assisted workflows sped up implementation, instrumentation, and repeated measurement.

Executive Summary

Firewood is an embedded key-value store for Merkleized blockchain state. It stores trie nodes directly on disk, instead of emulating it over a generic key-value store (e.g. LevelDB). This preserves fast iteration without compaction and supports EVM or other authenticated state[1].

This report uses Firewood replay: recorded C-Chain workloads are replayed in isolation, while re-execution is reserved for end-to-end confirmation; the report does not analyze broader EVM latency and uses recent 50M-77M block range. Reads account for 63% of Firewood time, cold node I/O accounts for 89.5% of read time in the 100,000-block trace, and missing reads consume twice the time share implied by their count. The bottleneck in this dataset is therefore the cold read path.

Replay began as a way to reproduce a persistent state-root mismatch ([Firewood PR #1557](#)) and became an isolated benchmark with metrics, profiles, and root checks. A rebuilt state preserves experiment rankings while reducing a five-run session from 57.2 to 12.4 minutes, enabling continuous per-change measurement on stable remote machines.

This work contributes a reusable benchmark suite: fixed 10k, 100k, and 1M ranges drawn from recent heavy workloads, versioned initial-state and replay-log artifacts, and an interleaved cold-copy A/B protocol. Root-verified candidates then graduate to C-Chain re-execution.

Table 1. Measured changes. Replay covers Firewood; re-execution covers EVM plus Firewood. Negative is faster.

Work / PR	Replay Δ	Re-exec. Δ	Status
#2201 , avoid deep clone	-2.02%	$\sim -1.31\%$	Merged
#2202 , evict update reads	-1.71%	$\sim -1.11\%$	Merged
#2203 , cap Merkle pool	-1.29%	$\sim -0.84\%$	Draft
#2204 , constant empty root	-0.82%	$\sim -0.53\%$	Merged
Four-PR shipping stack	-4.94%	$\sim -3.21\%$	3 merged + draft
#1963 , metric handles (DP=64)	-5.3%	$\sim -3.45\%$	Draft
#1961 , inline RLP hashes	$-(1.4-1.8)\%$	$\sim -(0.9-1.2)\%$	Draft
#5953 , warmup + lookahead	—	$-(11.7-13.0)\%$ E2E	Draft
#2248 , Bloom filter	-9.9% total	$\sim -6.4\%$	Prototype
Block access lists	—	$\sim -50\%$ E2E	Early estimate
In-kernel reads (XRP, io_uring/BPF)	No useful gain	No useful gain	Park

The shipping stack reduces replay time by 4.94%; metric-handle caching and RLP inlining bring the low-hanging end-to-end opportunity close to 10%. The [Bloom-filter prototype \(Firewood PR #2248\)](#) cuts replay time by 9.9% but moves some work into proposal. [Parallel warmup \(avalanche PR #5953\)](#) and lookahead improve end-to-end throughput by 13.2–15.0%, while in-kernel reads show no useful gain. A BAL-like experiment estimates roughly 50% lower end-to-end elapsed time: it needs replication and careful design to determine whether the distribution of BAL can work with Streaming Asynchronous Execution (SAE), as block-builder doesn't execute the blocks[3][2]. AI accelerated implementation and measurement; humans retained responsibility for workload validity, artifacts, and shipping decisions.

Contents

1	Introduction	1
2	Methodology	2
2.1	What Replay Measures	2
2.2	Time and Noise	2
2.3	AI-Based Optimization Workflow	3
3	Workload Analysis	4
3.1	Where the Time Goes	4
3.2	Operation Latency: Counts versus Time	5
3.3	Benchmark Ranges	6
3.4	From Aggregate Reads to Trie Structure	7
3.5	What the Structure Implies	7
3.6	Key-Value Distributions	8
3.7	Read Structure and Latency	8
3.8	Directions Exposed by the Workload	10
4	Low-Hanging Improvements	11
4.1	Four-PR Storage Stack	11
4.2	Metric Handle Caching	11
4.3	Inline RLP Hashes	11
5	Structural Performance Experiments	12
5.1	Bloom Filter	12
5.2	Parallel EVM Warmup and Lookahead	13
5.3	In-Kernel Reads: XRP and io_uring/BPF	13
5.4	Block Access Lists	13
6	New Tooling and Benchmarks	14
6.1	Canonical Ranges	14

6.2	Replay and CI	14
6.3	Dedicated Benchmark Machines (Possible Next Step)	14
7	Conclusion	15
	References	16

List of Figures

1	Replay workflow	3
2	AI-based optimization workflow	3
3	Firewood time by component	4
4	Operation latency distributions	5
5	Read-latency density	6
6	Selected 1M benchmark range	6
7	Trie pattern taxonomy	7
8	Key-value distributions	8
9	Read structure and latency	9
10	Cold I/O budget	10

SECTION 1

Introduction

Firewood is the performance-oriented database for C-Chain EVM state. This report identifies a representative workload, shortens the measurement loop, and tests the improvements most likely to matter.

C-Chain re-execution is the final end-to-end benchmark, but a throughput result alone does not show which Firewood operation owns latency or whether the chosen blocks represent current mainnet work. Replay isolates the recorded database calls, verifies the resulting state roots, and makes the same workload inexpensive to profile and repeat.

The workload analysis shows that newer ranges are heavier than the historical benchmark range and that reads dominate Firewood time. The report therefore proposes fixed 10k, 100k, and 1M ranges, re-evaluates low-hanging pull requests under one controlled protocol, and measures four structural directions: [Bloom filter \(Firewood PR #2248\)\[4\]](#), [parallel EVM warmup \(avalanchego PR #5953\)](#), in-kernel reads, and block access lists[3].

AI accelerated implementation and repeated A/B measurement after the workload and correctness oracle were fixed. Human review remained responsible for benchmark validity, system artifacts, and shipping decisions.

Contributions

- **Benchmark loop:** faster replay with root verification and stable experiment metadata;
- **Workload evidence:** representative ranges and a measured cold-read bottleneck;
- **AI workflow:** faster hypothesis-to-patch iteration under human-controlled workload selection, correctness checks, and shipping decisions;
- **Decision evidence:** comparable results for low-hanging pull requests and structural changes.

SECTION 2

Methodology

Replay records Firewood calls from C-Chain execution, runs them directly against the database, and verifies the resulting roots. It is a faster and quieter proxy for end-to-end performance work.

2.1 What Replay Measures

A replay log fixes the ordered reads, proposals, commits, and revision operations produced by C-Chain execution. The runner applies those operations without the EVM, verifies the resulting roots, and emits wall-clock measurements, metrics, traces, and flamegraphs. Candidate and baseline binaries run in interleaved order from fresh cold copies of the same state.

Replay is the iteration benchmark; C-Chain re-execution remains the final confirmation. Across the experiments in this report, replay preserved the direction and ranking of changes. Replay-only effects are converted to an end-to-end estimate with the observed factor of 0.65 and are labeled as estimates wherever reported. Figure 1 maps the full capture, replay, measurement, and confirmation path.

2.2 Time and Noise

Table 2. Five-run session cost, including state setup.

Method	State size	Session time	Speedup
C-Chain re-execution	443 GB	57.2 min	1.0×
Replay, original state	221 GB	29.8 min	1.9×
Replay, rebuilt state	53 GB	12.4 min	4.6×

Replay saves 27.4 minutes per five-run session compared with re-execution; rebuilding the same latest state saves another 17.4 minutes. Root verification keeps the runs comparable, and the smaller state preserves experiment rankings while reducing setup and I/O variance. Under the cold-copy protocol, replay's coefficient of variation is 0.09 versus 0.32 for re-execution.

METHOD

Use the rebuilt replay state for rapid, low-noise iteration, then confirm shipping candidates with end-to-end C-Chain re-execution.

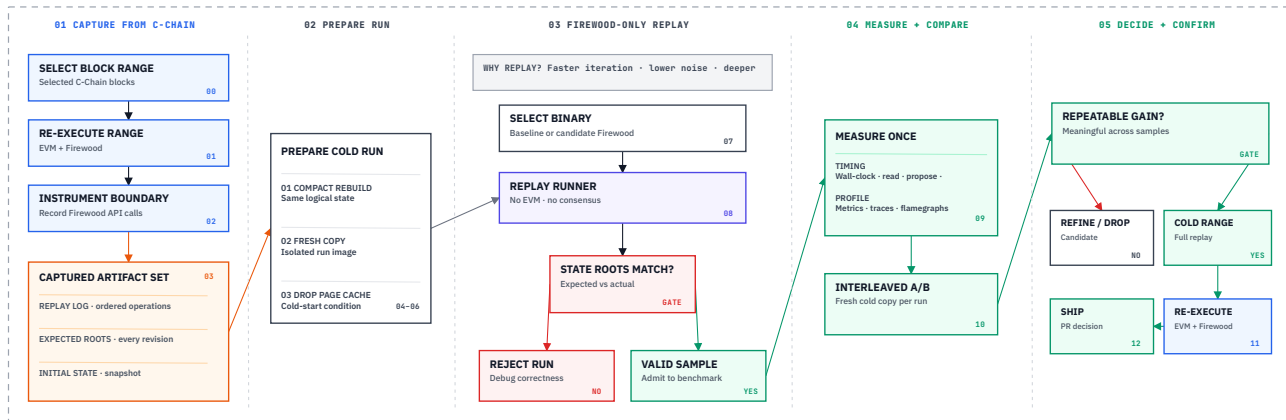


Figure 1. Replay workflow. C-Chain re-execution captures ordered Firewood operations, expected roots, and the initial state. Baseline and candidate binaries then replay fresh cold copies, reject any root mismatch, and enter interleaved measurement. Repeatable gains advance to a full cold replay and end-to-end re-execution before a shipping decision.

2.3 AI-Based Optimization Workflow

AI accelerated the mechanical part of the optimization loop: implementing a bounded hypothesis, running correctness-checked replay, comparing profiles and measurements, and iterating when the evidence justified another candidate. Human input could redirect the loop at any stage. Meaningful results entered a separate release path for human validation, cleanup, benchmark preservation, and the final shipping decision; insignificant results were checked and archived.

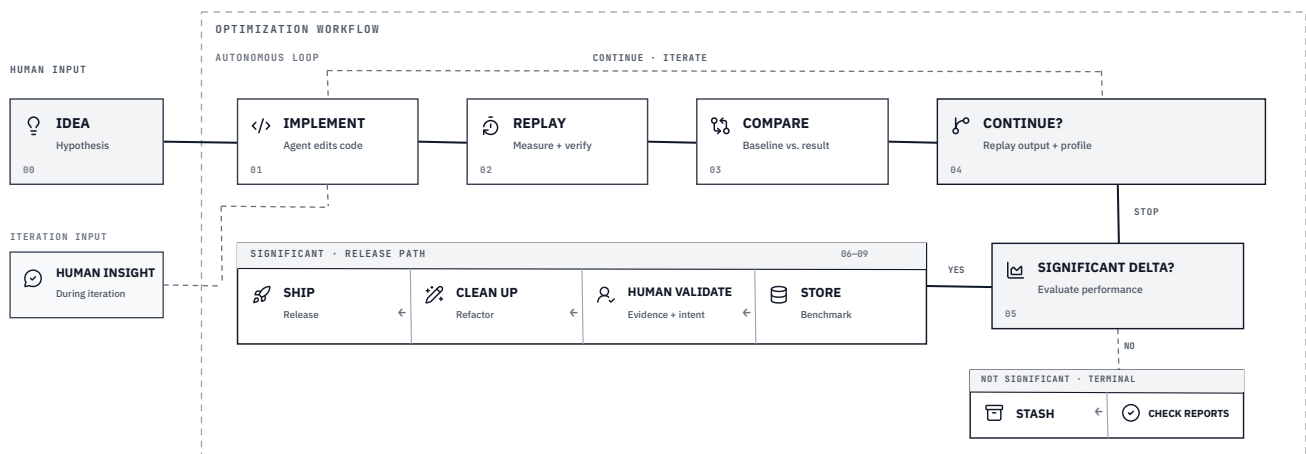


Figure 2. AI-based optimization loop used in this work. Automation shortens implementation and measurement cycles, while humans retain responsibility for the hypothesis, workload validity, evidence interpretation, correctness, and release decisions.

SECTION 3

Workload Analysis

A 27-million-block replay first locates the dominant operation and representative benchmark ranges; a focused 100,000-block trace then follows the read bottleneck into trie structure, outcomes, and cold I/O.

The analysis proceeds from broad to narrow. First, an operation-level trace over heights 50.0M–77.0M locates time by phase, measures the distributions hidden by aggregate throughput, and searches every rolling 10k, 100k, and 1M window for sustained heavy workloads. Second, after reads emerge as the dominant cost, state scans and a focused cold 100,000-block trace describe trie shape, key populations, cache and memory resolution, cold depth, and found-versus-missing outcomes. The result is both a bottleneck diagnosis and a benchmark specification.

The broad trace contains 27 million blocks and 7.374 billion Firewood operations, beginning at height 50,000,000. A multi-pass pipeline produces per-block series, exact per-bucket latency distributions, latency-density heatmaps, and exhaustive rolling-window rankings. Each full scan takes 30–130 seconds on 16 cores, so the analysis can be rerun as the questions or target ranges change.

3.1 Where the Time Goes

Reads account for 63% of all recorded Firewood time across the range, confirming the preliminary replay finding at much larger scale. Figure 3 splits per-block time into reads, ProposeOnDB, and commit plus other operations. The chart also marks the main workload eras: the IO-stall cluster around height 56.44M, the miss-heavy era at 60–62M, and the heaviest sustained era around 72M.

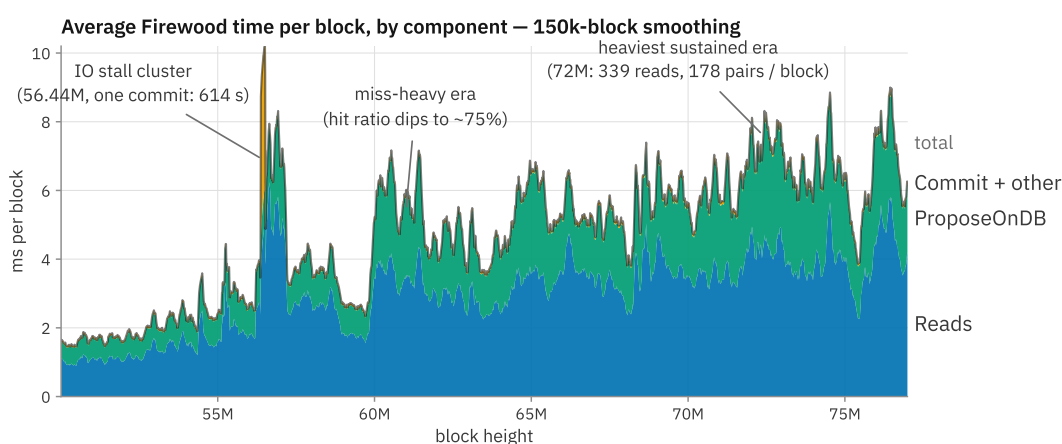


Figure 3. Per-block Firewood time by component (reads, ProposeOnDB, commit and other) over heights 50M–77M, with annotated events: the 56.44M IO-stall cluster, the 60–62M miss-heavy era, and the 72M heaviest sustained era.

3.2 Operation Latency: Counts versus Time

Percentile summaries answer how many operations are slow, but not how much the slow ones cost. For each operation type, the analysis therefore pairs the count distribution over duration buckets with the exact time contribution of each bucket, computed by a dedicated scan rather than estimated from bucket midpoints. Table 3 summarizes the result: for every major operation, a large share of total time is concentrated in a small fraction of slow calls.

Table 3. Operation latency and time concentration. The last three columns give the share of that operation’s total time carried by its slowest 10%, 1%, and 0.1% of calls.

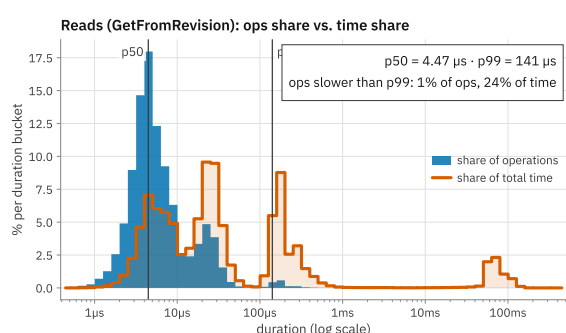
Operation	p50	p99	Time share in slowest...		
			10%	1%	0.1%
Reads (GetFromRevision)	4.5 μ s	141 μ s	47%	24%	10%
ProposeOnDB	891 μ s	17.8 ms	45%	14%	2.5%
Commit	35 μ s	141 μ s	46%	37%*	35%*
FreeRevision	0.35 μ s	56 μ s	50%	12%	2%
GetRevision	1.8 μ s	2.8 μ s	17%	4%	1%
RootHash	0.18 μ s	0.56 μ s	14%	2%	1%

* Commit’s tail share is dominated by a single operation: the 614 s stalled commit at block 56,443,215 accounts for 33% of all commit time in the trace.

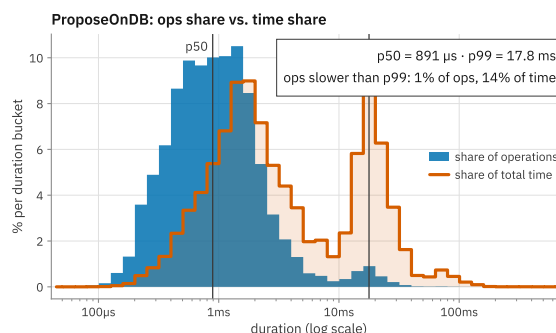
Three observations follow from the paired distributions. **Read time is trimodal**: operations cluster near 25 μ s for deeper in-memory work, between 150 and 300 μ s around p99, and in a rare 60–100 ms stall mode. The stall mode contains only 0.0002% of reads but about 4% of total read time. In contrast, **ProposeOnDB time is bimodal**: most operations take 1–3 ms, while a second mode near 18 ms carries about 10% of propose time. These read-latency modes remain stable across workload eras. Figure 5 shows persistent cache and disk bands, with p50 near 5 μ s throughout and p99 varying from 40 to 200 μ s by era.

TAIL CONCENTRATION

Read time lives in the tail: the slowest 10% of reads carry 47% of total read time. Halving the latency of only the tail beyond p90 would cut total read time roughly in half.



(a) Reads (GetFromRevision).



(b) ProposeOnDB.

Figure 4. Operation count distribution paired with time distribution over duration buckets. The count mode and the time mode disagree: most operations are fast, but most time belongs to the slow buckets—trimodal for reads, bimodal for proposals.

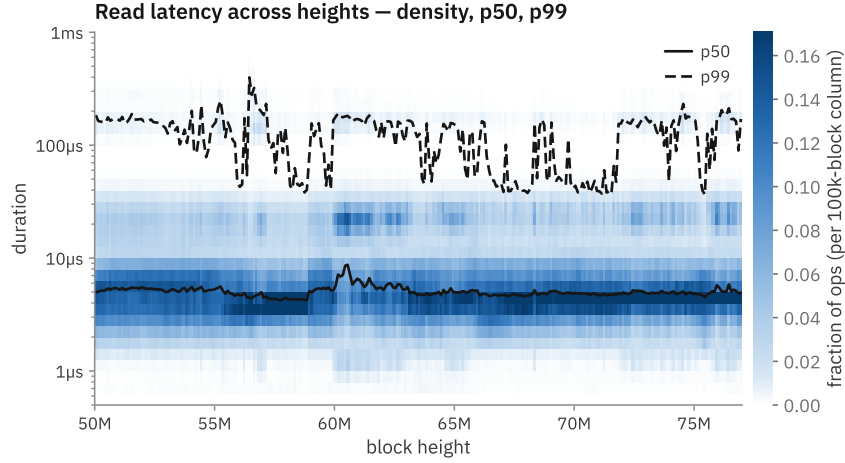


Figure 5. Read-latency density over heights. The discrete horizontal bands are the cache and disk modes; p50 stays near 5 μ s across the whole range while p99 swings between 40 and 200 μ s by era.

3.3 Benchmark Ranges

The independently strongest windows are 56,608,938–56,618,937 at 10k, 76,411,182–76,511,181 at 100k, and 71,973,865–72,973,864 at 1M. The maintained suite instead favors recent traffic and a single reusable starting state. Its 1M window, 75,853,064–76,853,063, is only 0.7% lighter than the overall winner and ranks first for read time. The shorter benchmarks are prefixes of that window:

- **10k:** heights 75,853,064–75,863,063 for quick pull-request checks;
- **100k:** heights 75,853,064–75,953,063 for validation runs;
- **1M:** heights 75,853,064–76,853,063 for sustained performance work.

All three ranges share height 75,853,064 as their starting state and use one master replay capture. The nested prefixes therefore cover quick checks, validation, and sustained work with one initial-state artifact while remaining in the selected recent read-heavy window.

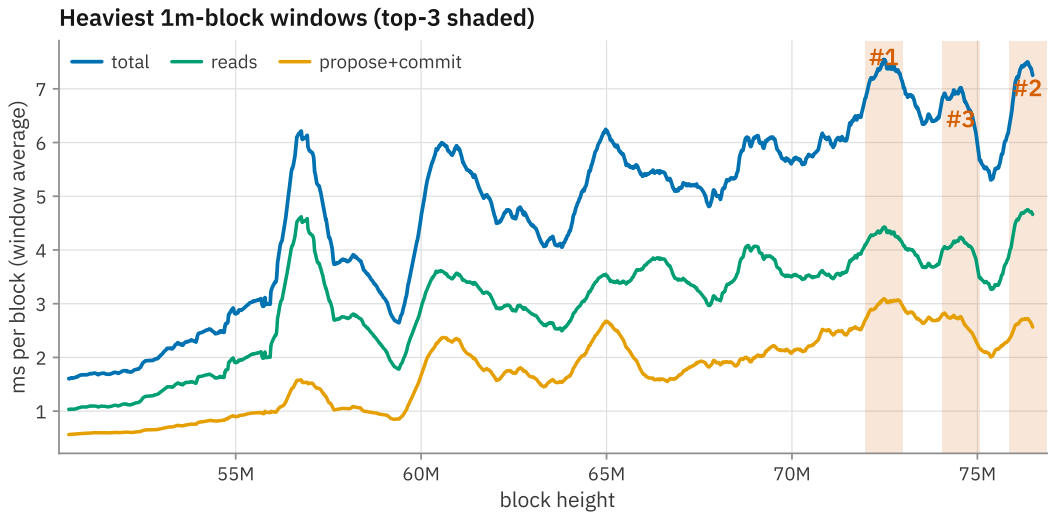


Figure 6. The selected 75.85M–76.85M range is the strongest read-heavy 1M window and is within 0.7% of the overall leader in the 50M–77M scan.

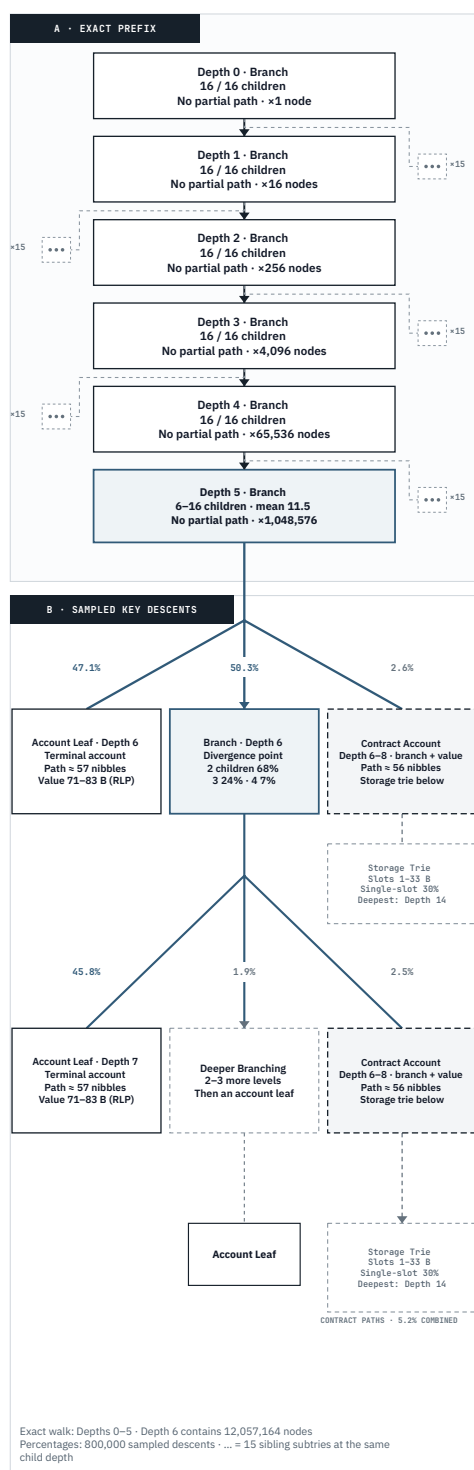


Figure 7. Trie pattern taxonomy from 800,000 sampled descents. The diagram separates the dense upper branch levels from the lower account, divergence, contract-account, and storage-slot path classes.

3.4 From Aggregate Reads to Trie Structure

Aggregate timings establish that reads dominate, but they do not explain why individual reads vary so widely. The next analysis follows each descent through the trie and separates the factors that determine its cost: path class, depth reached, cache locality, revision age, read outcome, and cold-node I/O. Figure 7 provides the structural map for that analysis.

To investigate reads more deeply and design a better benchmark that matches the observed distribution, the next step is a focused analysis of the trie structure. The benchmark should model the work Firewood actually performs: depth traversed, cache locality, revision age, cold reads, and the latency profile of each read class.

Structural reading. Read latency should be analyzed by path class, not only by aggregate depth. The important split is where a descent leaves the fully populated top structure and enters account, divergence, contract, or storage-slot behavior.

Benchmark implication. A representative benchmark should keep account leaves, divergence branches, contract paths, and storage-slot reads separate.

Questions. The trie analysis measures total and average traversal depth for a selected block range. It then separates reads that resolve in memory or cache from reads that touch cold nodes, accounts for revision locality, and identifies the path classes that contribute most to latency.

3.5 What the Structure Implies

The sample shows a dense, regular upper trie and sparser lower path classes. Benchmark design should preserve that distinction because it changes cache behavior and cold traversal cost.

3.6 Key-Value Distributions

Key and value populations matter because equal read counts can produce very different costs when one workload repeats warm paths and another forces cold traversal.

A full scan of the state at height 50M gives the static side of this picture. The database holds 343,895,712 key–value pairs: 21,274,984 account keys (32 bytes, 6.2%) and 322,620,728 storage-slot keys (64 bytes, 93.8%). Keys carry far more bytes than values: 21.3 GB of key material versus 4.8 GB of values. The mean key is 62.0 bytes, while the mean value is only 14.1 bytes.

The value-size histogram (Figure 8a) splits exactly along key type. Storage-slot values occupy 1–33 bytes, dominated by small encoded words (modes at 1, 3, 5, 21, and 33 bytes). Account records form an isolated 71–85-byte cluster whose 21,274,984 values match the account-key count exactly; nothing exists between 34 and 70 bytes. The trie therefore stores two cleanly separated value populations, and a benchmark can reproduce them with two draws instead of one blended distribution.

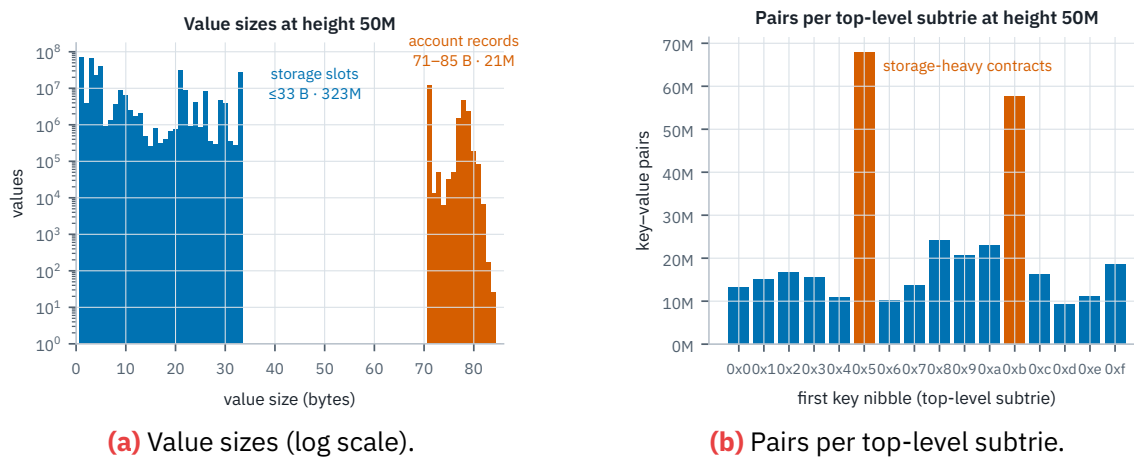


Figure 8. Key-value distributions from the full state scan at height 50M (343.9M pairs). Value sizes split exactly by key type: storage-slot values at 1–33 bytes, account records in an isolated 71–85-byte cluster. Per-nibble counts vary $7.3\times$ across the sixteen top-level subtries; the 0x5 and 0xb subtries alone hold 37% of all pairs.

The keyspace is hashed, so the first nibble is uniform over accounts; the $7.3\times$ spread in Figure 8b (9.3M pairs under 0xd against 68.0M under 0x5) is contract weight. Storage-heavy contracts concentrate 37% of all pairs under two of the sixteen top-level subtries, which means subtrie density—and with it node size, fanout, and cache pressure—is workload-shaped rather than uniform. A benchmark that samples keys uniformly over the keyspace will under-visit exactly the dense regions a real replay hits hardest.

3.7 Read Structure and Latency

Read latency should be analyzed by read type and trie traversal shape: depth reached, cache hit or miss status, whether the read was missing, and whether it caused disk activity. The first pass comes from a 100,000-block replay against the 221 GiB height-50M database. The run uses a fresh non-copy-on-write copy, deferred persistence every 1,000 blocks, a dropped page cache, and no negative filter, providing a clean cold-start baseline. The trace captures 7,701,766 `get_value` calls and 83,185,428 node visits, 10.8 nodes per read on average. Total in-read time is 237.7 s, of which 212.8 s—89.5%—is cold disk I/O.

READ-PATH FINDING

The read path is disk-bound: 89.5% of read time is cold node I/O, while cache and memory visits are effectively free. Fully 68% of reads never touch disk. Reads with three or four cold nodes are 17% of reads but 61% of the cold budget, while missing reads are 16% of reads but 32% of read time.

Table 4 shows where node visits resolve. Disk is only 8.9% of visits but essentially all of the latency: a cache hit costs 0.42 μ s, about 68 $\times$ cheaper than the 28.8 μ s average cold read. The optimization target is the 7.39M cold disk reads, not the CPU work around them.

Table 4. Node visits by source over the 100k-block trace. Disk resolves 8.9% of visits but carries essentially all of the read latency.

Source	Visits	Share	Mean latency	p99
Memory	57,025,549	68.5%	≈ 0 μ s	—
Cache	18,769,891	22.6%	0.42 μ s	0.67 μ s
Disk (cold)	7,389,988	8.9%	28.79 μ s	184.2 μ s

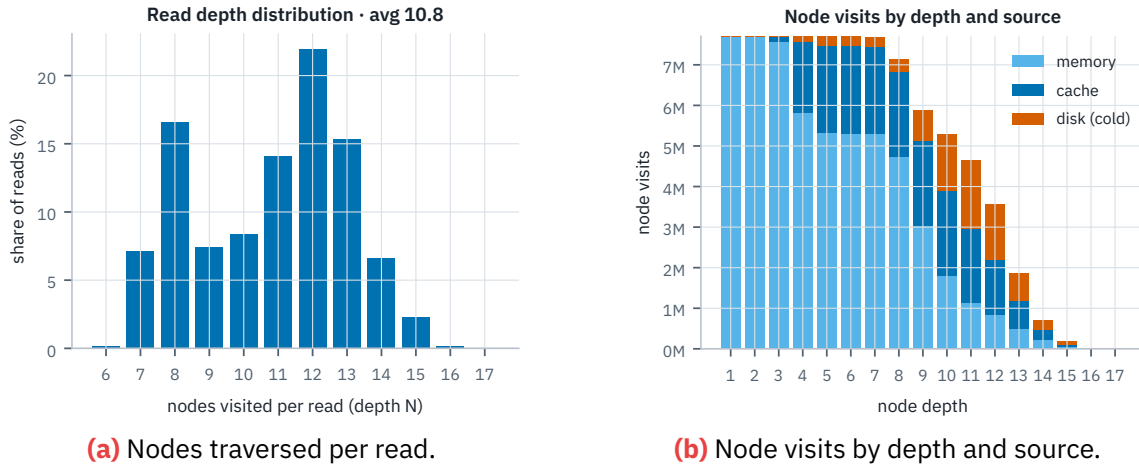


Figure 9. Read structure over the 100k-block trace. Reads traverse 10.8 nodes on average, concentrated at depths 8 and 11–13. The visit breakdown shows the shallow-warm, deep-cold shape: depths 1–3 resolve almost entirely in memory, while cold reads concentrate at depths 9–13.

The trace separates four read classes. First, **warm reads** account for 68.1% of requests and complete with zero cold nodes in about 3 μ s; 27.8% resolve entirely in memory. No request is purely cold because every descent begins in the memory-resident upper levels.

By contrast, **cold reads** are only 31.9% of requests but carry the entire 212.8 s cold budget. Cost grows by roughly 29 μ s per additional cold node, while per-node disk latency rises from 11 μ s at depth 4 to 46 μ s at depth 13 and 74 μ s at depth 17 (Figure 10b). Reads with three or four cold nodes consume 61% of the cold budget (Figure 10a), led by depth-12 and depth-13 shapes.

Within the cold class, **missing reads** are disproportionately expensive. They represent 16.2% of requests but 32.4% of read time and 34.4% of cold I/O (Table 5). A miss averages 61.7 μ s, compared with 24.9 μ s for a found read, because proving absence traverses about 11 nodes and typically incurs one to three cold reads.

Finally, **high-latency reads** are dominated by individual storage stalls rather than long traver-

sals. Each of the 25 slowest requests, reaching 29 ms, spends more than 99.9% of its time in one stalled disk read. The tail spans found and missing outcomes, so improving p99 and maximum latency requires better device-I/O behavior rather than a different traversal shape.

Table 5. Read time by outcome. Missing reads are $2.5\times$ more expensive than found reads and almost entirely cold disk time.

Outcome	Reads	% reads	% time	% cold I/O	Mean	p50	p99
Found	6,454,770	83.8%	67.6%	65.6%	24.9 μ s	3.2 μ s	361 μ s
Missing	1,246,996	16.2%	32.4%	34.4%	61.7 μ s	7.9 μ s	367 μ s

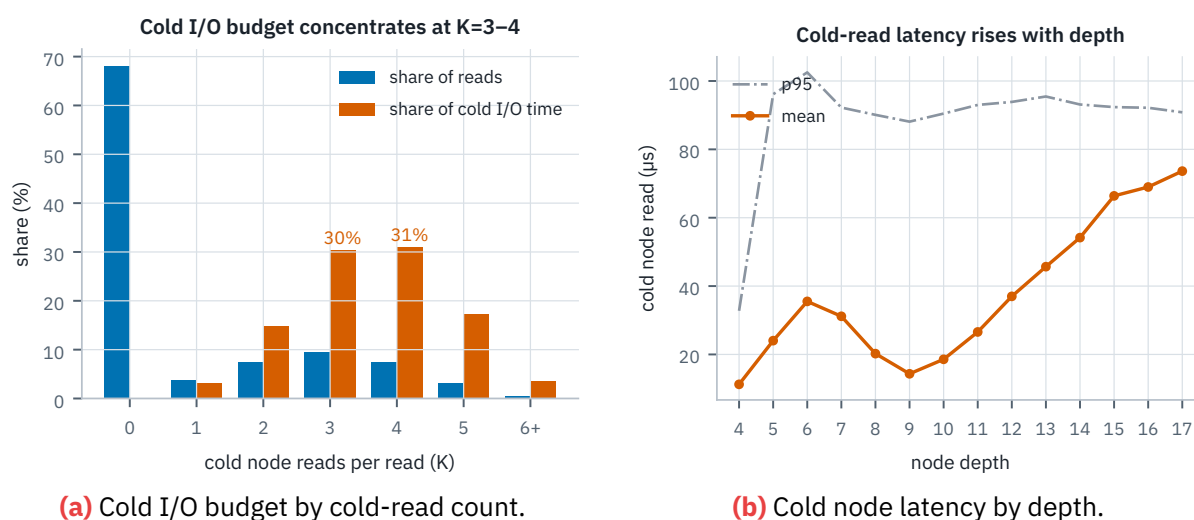


Figure 10. Where the cold budget sits. Reads paying three or four cold node reads are 17% of reads but 61% of all cold I/O time, and the cold reads themselves get slower as they go deeper—consistent with deep nodes scattering across the file.

The trace also bounds the value of warming missed keys instead of filtering them. Only 15.9% of misses are later read as found, at a median lag of 29,597 reads (p90 3.9M)—far beyond any realistic cache lifetime. Account-key misses are reused at 45.2% versus 14.7% for storage keys, but storage keys are 96% of all misses. Prefetching or negative-caching a missed key therefore has almost no future-read value. A membership filter that skips the walk entirely (Section 5.1) and better cold-read I/O (Section 5.3) attack the measured cost directly.

BENCHMARK IMPLICATION

A representative benchmark should preserve read distribution, not just replay volume. Depth traversed, cache status, revision age, and missing-read behavior are part of the workload.

3.8 Directions Exposed by the Workload

The measurements point to four practical experiments. A **Bloom filter** (Firewood PR #2248) can skip expensive missing-read traversals. **Parallel EVM warmup** (avalanche PR #5953) can issue likely reads before serial execution needs them. In-kernel reads can test whether reducing syscall and traversal overhead matters. A BAL-like structure can expose reads early for parallel I/O, but must not depend on a block builder. The next two sections report each change, its measured result, and the resulting decision.

SECTION 4

Low-Hanging Improvements

Small, localized code changes can improve performance without changing the architecture. Replay makes these low-hanging gains measurable and shows when several modest wins add up.

Each candidate ran from a fresh cold copy, in interleaved baseline and candidate rounds, with recorded roots used as the correctness oracle.

Table 6. Low-hanging performance pull requests. Negative Δ reduces elapsed time.

Work / PR	Replay Δ	Re-exec. est.	Status
#2201, avoid deep clone	−2.02%	−1.31%	Merged
#2202, evict update reads	−1.71%	−1.11%	Merged
#2203, cap Merkle pool	−1.29%	−0.84%	Draft
#2204, constant empty root	−0.82%	−0.53%	Merged
Four-PR shipping stack	−4.94%	−3.21%	3 merged + draft
#1963, metric handles (DP=64)	−5.3%	−3.45%	Draft
#1961, inline RLP hashes	−(1.4–1.8)%	−(0.9–1.2)%	Draft

4.1 Four-PR Storage Stack

Change. PR #2201 stops deep-cloning unhashed child nodes during classification, removing an allocation and copy from the hot path. PR #2202 evicts a node from the read cache when it is read for update, avoiding duplicate ownership of a node that is about to change. PR #2203 caps the Merkle worker pool at the machine’s available parallelism, preventing oversubscription and scheduling overhead. PR #2204 replaces repeated computation of the empty-trie root with its constant value.

Numbers. The individual replay effects range from −0.82% to −2.02%. Together they reduce replay time by 4.94% ($t = -14.25$, six runs), an estimated 3.21% end-to-end reduction. Three changes are merged; the Merkle pool cap remains draft.

4.2 Metric Handle Caching

Change. PR #1963 resolves metric handles once after recorder installation and reuses them on the persist and allocation paths instead of repeating registry lookups.

Numbers. With metrics enabled on both sides, replay improves by 4.2% on one state and 5.3% on the second ($t = -4.1$ and -3.0 , five runs per cell). At the avalanche persistence setting, metrics overhead falls from 5.7% to about 1.4%.

4.3 Inline RLP Hashes

Change. PR #1961 stores small RLP hash payloads inline instead of routing them through a heap-backed collection, removing allocation and indirection from repeated node encoding[5].

Numbers. The change reduces replay time by 1.4% and 1.8% across two states, with five runs per state and identical verified roots.

SECTION 5

Structural Performance Experiments

These experiments change how reads are avoided, issued, or overlapped. They require architectural work, but replay makes their costs, correctness, and end-to-end potential measurable.

The experiments use fresh cold copies, interleaved baseline and candidate runs, and a gas or state-root correctness oracle. Results measured only in replay remain labeled as end-to-end estimates.

Table 7. Structural experiment decisions.

Architectural change	Best measured result	Take
#2248, Bloom filter	reads -25.1% ; replay total -9.9%	Prototype; proposal becomes limiting
#5953, warmup and lookahead	$+15.0\%$ (25k); $+13.2\%$ (100k)	Draft
In-kernel reads	No useful end-to-end gain	Park; redesign needed
Block access lists	$\sim -50\%$ E2E on five warm 10k runs	Early estimate; design for SAE

5.1 Bloom Filter

Implementation. The experiment is available as [Firewood PR #2248](#), *feat: bloom filter experimental prototype*.

Architectural change. A counting Bloom filter checks key membership before trie traversal. It loads from a durable checkpoint, tracks inserts at commit, and falls back to the canonical walk on a positive result[4].

Numbers. On the 100k-block replay, the filter skips 1.16 million missing-key walks. Read time falls from 243.3 to 182.3 seconds (-25.1%); proposal rises from 43.0 to 75.6 seconds because skipped reads no longer warm later inserts; total time falls from 288.7 to 260.2 seconds (-9.9%). Verification found no false negatives across 7.7 million reads.

Table 8. Bloom filter result on the 100k-block cold replay.

Configuration	Reads	Propose	Total	Change
Baseline	243.3 s	43.0 s	288.7 s	—
Bloom filter	182.3 s	75.6 s	260.2 s	-9.9%

TAKE

The filter safely removes a real miss cost, but part of the work moves into proposal. Continue the [experimental prototype](#) and evaluate it by total time, not read time alone.

5.2 Parallel EVM Warmup and Lookahead

Implementation. Intra-block warmup is available as draft [avalanche PR #5953](#), *perf: intra-block speculative warmup of transaction state*. Lookahead remains the related next-block extension measured alongside it in this report.

Architectural change. Spare cores execute transactions against throwaway state to discover and warm reads before the serial execution path needs them. Warmup works within the current block; lookahead starts on the next known block.

Numbers. On the 25k cold range, warmup alone improves throughput by 10.2%, lookahead alone by 8.7%, and warmup plus lookahead by 15.0%. The combined result reproduces at 13.2% over 100k blocks.

Table 9. Parallel EVM result on the 25k cold range.

Configuration	Mgas/s	Change
Baseline	68.92	—
Warmup	75.95	+10.2%
Lookahead	74.93	+8.7%
Warmup + lookahead	79.29	+15.0%

TAKE

Continue the draft [intra-block warmup PR](#) and confirm its gain on the current representative ranges; retain lookahead as the related experimental extension.

5.3 In-Kernel Reads: XRP and io_uring/BPF

Architectural change. XRP and io_uring/BPF move parts of trie traversal or dependent-read submission into the kernel to reduce userspace round trips and create I/O queue depth[6][7][8].

Numbers. Correctness-verified reruns show no useful end-to-end gain. Firewood’s hot upper levels already resolve from memory and cache, leaving too little syscall overhead for the offload to repay its added path complexity.

TAKE

Park the prototypes. Revisit only with a storage design that batches cold dependent reads while preserving Firewood’s cache authority.

5.4 Block Access Lists

Architectural change. The experiment records a BAL-like read set and uses a bounded worker pool to issue those reads in parallel before execution. It models early read availability without assuming that a block builder supplies a protocol access list[3].

Numbers. Five warm 10k-block runs estimate roughly 50% lower end-to-end elapsed time. This intentionally preliminary estimate from one range is not a shipping claim.

TAKE

Treat the result as an estimate and replicate it. Any protocol design must work with Streaming Asynchronous Execution (SAE)[2] without relying on the block builder; serialization and resource limits follow only after that constraint is resolved.

SECTION 6

New Tooling and Benchmarks

The reusable output is a compact benchmark loop: fixed ranges, correctness-checked replay, and dedicated machines for stable runs.

6.1 Canonical Ranges

All three benchmarks begin at height 75,853,064. The 10k range ends at 75,863,063, the 100k range ends at 75,953,063, and the 1M range ends at 76,853,063. These nested prefixes use one master replay capture and one initial state for quick pull-request checks, validation runs, and sustained experiments in the recent 75M–76M era.

6.2 Replay and CI

The replay runner isolates Firewood calls, verifies recorded roots, and emits phase totals, metrics, traces, flamegraphs, and the metadata needed to compare runs. Rebuilt and compact state artifacts reduce setup cost. CI runs baseline and candidate cells serially from fresh copies; meaningful results graduate to the full cold range and then C-Chain re-execution.

6.3 Dedicated Benchmark Machines (Possible Next Step)

Snoopy and Linus can provide controlled clone and cache policy without provisioning costly AWS machines only for stable performance work. A small remote interface can list datasets, start a versioned replay, report status, and return summaries or profiles. GitHub CI can dispatch expensive replay jobs to these dedicated machines instead of the general self-hosted runners.

SECTION 7

Conclusion

Firewood is already a stable, high-performance database. The remaining gains come from reducing cold reads and exposing parallel work.

Across heights 50M–77M, reads consume 63% of recorded Firewood time. The focused 100,000-block trace locates that cost in cold lower-trie traversal: cold node I/O consumes 89.5% of read time, while missing reads consume twice the time share implied by their request count.

The four-PR storage stack reduces replay time by 4.94%. Metric-handle caching and inline RLP hashes add further measured gains, bringing the low-hanging end-to-end opportunity close to 10%.

The structural results are selective. The [Bloom-filter prototype \(Firewood PR #2248\)](#) cuts read time by 25.1% and replay total by 9.9%, although some work moves into proposal. [Parallel warmup \(avalanchego PR #5953\)](#) and lookahead improve end-to-end throughput by 13.2–15.0%. In-kernel traversal does not help the current cache-rich path. A BAL-like experiment provides an early estimate of roughly 50% lower end-to-end elapsed time, not a shipping claim; it requires replication and a design that works with SAE without depending on a block builder to produce the read set.

The practical sequence is to adopt replay CI and the representative ranges, land validated low-risk branches, continue validation of [Bloom-filter PR #2248](#) and the draft [parallel-warmup PR](#), and replicate the access-list result. AI shortens this loop, while humans remain responsible for workload relevance, artifacts, correctness boundaries, and shipping decisions.

FINAL TAKE

Use the benchmark system established here to pursue structural read parallelism and execution overlap, while keeping shipping claims tied to current-range, end-to-end confirmation.

References

- [1] Ava Labs, *Firewood Database*, Avalanche Builder Hub. <https://build.avax.network/docs/nodes/architecture/execution/firewood>
- [2] Avalanche Foundation, *ACP-194: Streaming Asynchronous Execution*, protocol proposal. The architecture is now documented as Continuous Execution. <https://github.com/avalanche-foundation/ACPs/blob/main/ACPs/194-streaming-asynchronous-execution/README.md>
Current architecture overview: <https://build.avax.network/docs/nodes/architecture/execution/continuous-execution>
- [3] T. Wahrstätter et al., *EIP-7928: Block-Level Access Lists*, Ethereum Improvement Proposals, 2025. <https://eips.ethereum.org/EIPS/eip-7928>
- [4] B. H. Bloom, “Space/time trade-offs in hash coding with allowable errors,” *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970. <https://doi.org/10.1145/362686.362692>
- [5] Ethereum Foundation, *Recursive-length prefix (RLP) serialization*. <https://ethereum.org/developers/docs/data-structures-and-encoding/rlp/>
- [6] Y. Zhong et al., “XRP: In-Kernel Storage Functions with eBPF,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*, pp. 375–393, 2022. <https://www.usenix.org/conference/osdi22/presentation/zhong>
- [7] Linux man-pages project, *io_uring(7) – Linux manual page*. https://man7.org/linux/man-pages/man7/io_uring.7.html
- [8] Linux kernel project, *BPF Documentation*. <https://docs.kernel.org/bpf/>